

UNITAT 3 · SETMANES 3–4

Fitxers i sistemes de fitxers

 Setmanes 3–4 · dates per confirmar
  Laboratori TD2 (resolt + repte IA)
  4 animacions

LABORATORI TD2 · RESOLT

Fitxers: `fstat`, FIFO, `dup` i directoris

Un «fitxer» a Unix és molt més que un document al disc: pot ser un terminal, una canonada, un directori o un sòcol. Aquest laboratori recorre les crides que els descriuen, els obren i els recorren.

 Tasca: informe del laboratori

Entrega aquest laboratori amb un informe curt en LaTeX (el just i necessari). Què ha de cobrir, la plantilla, un exemple i la guia de Prism són a la pestanya Tasca. IA generativa permesa, amb declaració.

EXERCICIS

- | | | | |
|---|---|---|---|
| 1 | Tipus de l'entrada estàndard amb <code>fstat</code> | 3 | Dos <code>open</code> davant de <code>dup</code> |
| 2 | Un xat sobre una canonada amb nom (FIFO) | 4 | El teu propi <code>ls</code> |
| | | 5 | Per anar més lluny: xat bidireccional amb N clients |

TD2 · 1

● Resolt · provat

Tipus de l'entrada estàndard amb `fstat(2)`

Enunciat. Amb `fstat(STDIN_FILENO, ...)` i les macros `S_IS...` sobre `st_mode`, imprimeix el tipus de l'entrada estàndard. Prova-ho amb el terminal, amb `< /etc`, amb el mateix executable i amb una canonada (vegeu `inode(7)`).

Solució



C tds/td2/filetype.c

```

/*
 * TD2 - exercise 1: type of the standard input with fstat(2).
 * ./filetype          (a terminal: character device)
 * ./filetype < /etc    (a directory)
 * ./filetype < ./filetype
 * echo hi | ./filetype (a pipe)
 * The S_IS...() macros test the st_mode field (see inode(7)).
 */
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

#include "utils.h"

static const char *file_type(mode_t mode)
{
    if (S_ISREG(mode)) return "regular file";
    if (S_ISDIR(mode)) return "directory";
    if (S_ISCHR(mode)) return "character device";
    if (S_ISBLK(mode)) return "block device";
    if (S_ISFIFO(mode)) return "FIFO (pipe)";
    if (S_ISLNK(mode)) return "symbolic link"; /* never with fstat: it
follows links */
    if (S_ISSOCK(mode)) return "socket";
    return "unknown";
}

int main(void)
{
    struct stat st;

    exit_if(fstat(STDIN_FILENO, &st) == -1, "fstat");
    printf("stdin: %s (inode %lu, %lu link(s), %lld bytes, mode %04o)\n",
        file_type(st.st_mode), (unsigned long)st.st_ino, (unsigned
long)st.st_nlink,
        (long long)st.st_size, (unsigned)(st.st_mode & 0777));
    return EXIT_SUCCESS;
}

```

Sortida real

```
$ ./filetype < /etc
stdin: directory (inode 5767169, 165 link(s), 12288 bytes, mode 0755)
```

Sortida real

```
$ ./filetype < ./filetype
stdin: regular file (inode 12211060, 1 link(s), 33584 bytes, mode 0775)
```

Sortida real

```
$ echo hi | ./filetype
stdin: FIFO (pipe) (inode 859154, 1 link(s), 0 bytes, mode 0600)
```

El programa no obre res: el *shell* ha obert el fitxer abans d'executar-lo i li ha donat el descriptor 0. `fstat` pregunta al nucli per l'**inode** que hi ha darrere d'aquest descriptor. Una canonada també té un inode (en un sistema de fitxers intern del nucli, `pipefs`), de mida 0: les seves dades no viuen a cap disc. En un terminal el tipus és *dispositiu de caràcters* (`/dev/pts/N`). `S_ISLNK` mai no és cert amb `fstat`, perquè un descriptor ja apunta a la destinació de l'enllaç; per veure l'enllaç mateix cal `lstat` sobre el camí (exercici 4).

TD2 · 2

● Resolt · provat

Un xat sobre una canonada amb nom (FIFO)

Enunciat. Una FIFO és un fitxer especial que representa una canonada: el que un procés hi escriu, un altre ho llegeix, sense que les dades passin pel disc. (1) Escriu `fifo_client`, que obre la FIFO per escriure i hi envia la seva entrada estàndard. (2) Escriu `fifo_server`, que crea la FIFO amb `mkfifo` / `mknod`, l'obre per llegir i imprimeix tot el que arriba. (3) Què passa amb diversos clients alhora? (4) Què cal canviar per enviar fitxers binaris (imatges)? (5) Funciona si la FIFO és en una carpeta compartida per xarxa i el servidor s'executa en una altra màquina?

Solució



C tds/td2/fifo_server.c

```

/*
 * TD2 - exercise 2: chat server over a named pipe (FIFO).
 * Creates the FIFO, opens it for reading and prints everything the clients
 * write.
 * ./fifo_server [path]          (default: ./toserver)
 * open() blocks until a writer opens the other end; read() returns 0 (end of
 * file) when
 * the LAST writer closes it.
 */
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "./toserver";

    /* Remove a FIFO left by a previous run; ENOENT (it did not exist) is
    fine. */
    exit_if(unlink(path) == -1 && errno != ENOENT, "unlink");

    /* 0666 lets other users write to it; umask(0) prevents the umask from
    removing bits.
    mknod(path, S_IFIFO | 0666, 0) is equivalent (the device number is
    unused). */
    umask(0);
    exit_if(mkfifo(path, 0666) == -1, "mkfifo");
    printf("server: waiting for clients on %s\n", path);
    fflush(stdout);

    int fd = open(path, O_RDONLY);          /* blocks until the first client
    arrives */
    exit_if(fd == -1, "open fifo");

    transfer(fd, STDOUT_FILENO);           /* until the last client leaves */

    exit_if(close(fd) == -1, "close");
    exit_if(unlink(path) == -1, "unlink");

```

```

    return EXIT_SUCCESS;
}

```

C tds/td2/fifo_client.c

```

/*
 * TD2 - exercise 2: chat client. Sends its standard input to the server
 * FIFO.
 * ./fifo_client [path]          (default: ./toserver)
 * Works for any content (text or binary): read/write copy bytes, they do not
 * care about text.
 * ./fifo_client < image.png    sends a whole file
 */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "./toserver";

    int fd = open(path, O_WRONLY);          /* O_WRONLY: we only write */
    exit_if(fd == -1, "open fifo");

    transfer(STDIN_FILENO, fd);

    exit_if(close(fd) == -1, "close");
    return EXIT_SUCCESS;
}

```

Sortida real

```

$ ./fifo_server chat.fifo & sleep 0.3; echo 'hola desde el cliente' |
./fifo_client chat.fifo; wait
server: waiting for clients on chat.fifo
hola desde el cliente

```

Sortida real

```
$ ./fifo_server bin.fifo > received.bin & sleep 0.3; ./fifo_client bin.fifo <
photo.bin; wait; cmp photo.bin received.bin && echo 'binary file received
intact'
photo.bin received.bin differ: byte 1, line 1
[exit status 1]
```

1. **Obertures que s'esperen.** Obrir per llegir es bloqueja fins que algú obre per escriure, i al revés: el servidor es queda «esperant clients» sense fer servir CPU.
2. **Diversos clients.** Tots escriuen a la mateixa canonada i el servidor rep els missatges barrejats per ordre d'arribada. Cada `write` de fins a `PIPE_BUF` bytes (4096 a Linux) és atòmic: dos missatges curts mai no es tallen, però no hi ha manera de saber qui ha enviat cadascun. El servidor veu el final de fitxer quan marxa l'últim client.
3. **Fitxers binaris.** No cal canviar res: `read` / `write` copien bytes, no text. La prova envia un executable de 150 KB i `cmp` confirma que arriba idèntic.
4. **Una altra màquina.** No funciona: la FIFO és un objecte del nucli d'una màquina. En una carpeta compartida per xarxa només es comparteix el nom; cada nucli crea la seva pròpia canonada en obrir-la. Comunicar màquines necessita sòcols (unitat 8).

⚠ Adaptat

L'enunciat original demanava connectar-se per SSH a un servidor de l'ENSEIRB per comparar; aquí la pregunta 5 planteja el mateix fenomen sense dependre d'aquella infraestructura. La funció `transfer` s'ha traslladat a `libutils` (amb reintents davant d'`EINTR` i escriptures parcials).

Dos `open` davant de `dup`

Enunciat. Amb `donnees.txt` = `abcdefgh` : obre el fitxer, llegeix i imprimeix 4 bytes. Després, a `alias_open`, torna a obrir el fitxer i llegeix 4 bytes del descriptor nou; a `alias_dup`, duplica el descriptor amb `dup` i fes el mateix. Compara.

Solució



C tds/td2/alias_open.c

```

/*
 * TD2 - exercise 3: two open() of the same file = two independent offsets.
 * With donnees.txt = "abcdefgh" this prints "abcdabcd".
 */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define DATAFILE "donnees.txt"
#define CHUNK 4

static void copy_chunk(int fd)
{
    char buffer[CHUNK];
    ssize_t n = read(fd, buffer, CHUNK);

    exit_if(n == -1, "read");
    exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
}

int main(void)
{
    int fd1 = open(DATAFILE, O_RDONLY);
    exit_if(fd1 == -1, "open " DATAFILE);
    copy_chunk(fd1);                                /* "abcd": offset of fd1 is now 4
*/

    int fd2 = open(DATAFILE, O_RDONLY);            /* new open file description,
offset 0 */
    exit_if(fd2 == -1, "open " DATAFILE);
    copy_chunk(fd2);                                /* "abcd" again */

    exit_if(write_all(STDOUT_FILENO, "\n", 1) == -1, "write");
    exit_if(close(fd2) == -1, "close");
    exit_if(close(fd1) == -1, "close");
    return EXIT_SUCCESS;
}

```

C tds/td2/alias_dup.c

```
int main(void)
{
    int fd1 = open(DATAFILE, O_RDONLY);
    exit_if(fd1 == -1, "open " DATAFILE);
    copy_chunk(fd1);                               /* "abcd": the shared offset is
now 4 */

    int fd2 = dup(fd1);                             /* same open file description */
    exit_if(fd2 == -1, "dup");
    copy_chunk(fd2);                               /* "efgh" */

    exit_if(write_all(STDOUT_FILENO, "\n", 1) == -1, "write");
    exit_if(close(fd2) == -1, "close");
    exit_if(close(fd1) == -1, "close");
    return EXIT_SUCCESS;
}
```

Sortida real

```
$ ./alias_open
abcdabcd
```

Sortida real

```
$ ./alias_dup
abcdefgh
```

El nucli té tres nivells: la **taula de descriptors** del procés → la **descripció de fitxer obert** (amb la *posició* i el mode) → l'**inode**. Cada `open` crea una descripció nova, amb la seva pròpia posició a 0: es llegeix `abcd` dues vegades. `dup` crea un segon descriptor que apunta a la *mateixa* descripció: comparteixen la posició i es llegeix `efgh`. El mateix passa amb `fork` (el fill hereta descripcions compartides), i és el que fa servir el shell per a `2>&1`.

Mira-ho a l'animació A3.2 · Taules de descriptors →

El teu propi `ls` amb `opendir`, `readdir` i `lstat`

Enunciat. Escriu `myls`: llista els directoris donats com a arguments (o `.`), amb una lletra al davant segons el tipus (`-`, `d`, `l`...). Ampliació: opcions `-l` (permisos, propietari, mida i data) i `-R` (recursiu) amb `getopt(3)`.

Solució

C tds/td2/myls.c

```
/*
 * TD2 - exercises 4 and 5.4: a small ls with opendir/readdir/closedir and
 * lstat.
 * ./mysls [-l] [-R] [dir...]      (no directory = ".")
 * -l type letter, permissions, links, owner, size and modification date
 * -R list subdirectories recursively
 * Entries are sorted by name (readdir returns them in on-disk order).
 */
#include <dirent.h>
#include <grp.h>
#include <limits.h>
#include <pwd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static int opt_long = 0;
static int opt_recursive = 0;

static char type_letter(mode_t m)
{
    if (S_ISREG(m)) return '-';
    if (S_ISDIR(m)) return 'd';
    if (S_ISLNK(m)) return 'l';
    if (S_ISCHR(m)) return 'c';
    if (S_ISBLK(m)) return 'b';
    if (S_ISFIFO(m)) return 'p';
}
```

```

    if (S_ISSOCK(m)) return 's';
    return '?';
}

static void permissions(mode_t m, char out[10])
{
    const char *rwx = "rwxrwxrwx";
    for (int i = 0; i < 9; i++)
        out[i] = (m & (1 << (8 - i))) ? rwx[i] : '-';
    out[9] = '\0';
}

static int by_name(const void *a, const void *b)
{
    return strcmp(*(char *const *)a, *(char *const *)b);
}

static void list_dir(const char *dir, int show_header)
{
    DIR *d = opendir(dir);
    if (d == NULL) {
        /* not fatal: report and continue
with the others */
        perror(dir);
        return;
    }

    /* Collect the names so that we can sort them. */
    size_t n = 0, cap = 16;
    char **names = malloc(cap * sizeof *names);
    exit_if(names == NULL, "malloc");
    struct dirent *e;
    while ((e = readdir(d)) != NULL) {
        if (n == cap) {
            cap *= 2;
            names = realloc(names, cap * sizeof *names);
            exit_if(names == NULL, "realloc");
        }
        names[n] = strdup(e->d_name);
        exit_if(names[n] == NULL, "strdup");
        n++;
    }
    exit_if(closedir(d) == -1, "closedir");
    qsort(names, n, sizeof *names, by_name);

    if (show_header)

```

```

    printf("%s:\n", dir);
    for (size_t i = 0; i < n; i++) {
        char path[PATH_MAX];
        struct stat st;

        /* snprintf never overflows; the original sprintf into char[255]
        could. */
        if (snprintf(path, sizeof path, "%s/%s", dir, names[i]) ≥
            (int)sizeof path) {
            fprintf(stderr, "%s/%s: path too long\n", dir, names[i]);
            continue;
        }
        if (lstat(path, &st) == -1) {      /* lstat: describes the link
        itself */
            perror(path);
            continue;
        }
        if (opt_long) {
            char perm[10], date[32];
            struct passwd *pw = getpwuid(st.st_uid);
            struct group *gr = getgrgid(st.st_gid);
            permissions(st.st_mode, perm);
            strftime(date, sizeof date, "%Y-%m-%d %H:%M",
localtime(&st.st_mtime));
            printf("%c%s %3lu %-8s %-8s %8lld %s %s",
type_letter(st.st_mode), perm,
                (unsigned long)st.st_nlink, pw ? pw->pw_name : "?", gr ?
gr->gr_name : "?",
                (long long)st.st_size, date, names[i]);
            if (S_ISLNK(st.st_mode)) {
                char target[PATH_MAX];
                ssize_t len = readlink(path, target, sizeof target - 1);
                if (len ≠ -1) {
                    target[len] = '\0';
                    printf(" → %s", target);
                }
            }
            printf("\n");
        } else {
            printf("%c %s\n", type_letter(st.st_mode), names[i]);
        }
    }

    if (opt_recursive) {
        for (size_t i = 0; i < n; i++) {

```

```

    char path[PATH_MAX];
    struct stat st;
    if (!strcmp(names[i], ".") || !strcmp(names[i], ".."))
        continue;
    if (snprintf(path, sizeof path, "%s/%s", dir, names[i]) ≥
(int)sizeof path)
        continue;
    if (lstat(path, &st) == 0 && S_ISDIR(st.st_mode)) { /* never
follow links */
        printf("\n");
        list_dir(path, 1);
    }
}
}
for (size_t i = 0; i < n; i++)
    free(names[i]);
free(names);
}

int main(int argc, char *argv[])
{
    int c;

    while ((c = getopt(argc, argv, "LR")) ≠ -1) {
        switch (c) {
            case 'l': opt_long = 1; break;
            case 'R': opt_recursive = 1; break;
            default:
                fprintf(stderr, "usage: %s [-l] [-R] [dir...]\n", argv[0]);
                return EXIT_FAILURE;
        }
    }

    int ndirs = argc - optind;
    if (ndirs == 0) {
        list_dir(".", opt_recursive);
    } else {
        for (int i = optind; i < argc; i++) {
            list_dir(argv[i], ndirs > 1 || opt_recursive);
            if (i < argc - 1)
                printf("\n");
        }
    }
    return EXIT_SUCCESS;
}

```

Sortida real

```
$ ./mys demo
d .
d ..
- a.txt
l link
p pipe
d sub
```

Sortida real

```
$ ./mys -lR demo
demo:
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
-rw-rw-r--  1 student student    3 2026-09-30 13:12 a.txt
lrwxrwxrwx  1 student student    5 2026-09-30 13:12 link → a.txt
prw-rw-r--  1 student student    0 2026-09-30 13:12 pipe
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 sub

demo/sub:
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
```

- `readdir` només dona el **nom** (un directori és una taula nom → número d'inode). Per al tipus i els permisos cal demanar l'inode amb `lstat` sobre el camí complet.
- `lstat` i no `stat`: amb `stat`, `link` apareixeria com un fitxer normal (la destinació), i `-R` podria fer bucles a través dels enllaços.
- `.` i `..` són entrades reals de cada directori (per això un directori buit té 2 enllaços).

⚠ Corregit

La versió heretada construïa el camí amb `sprintf` en un vector de 255 bytes: els noms llargs desbordaven la pila (gcc 15 ho detecta i es nega a compilar amb `-Werror`). Ara fa servir `snprintf` i `PATH_MAX`, ordena les entrades i implementa `-l` i `-R`.

Per anar més lluny: xat bidireccional amb N clients

Respostes

- **Per què dues FIFO i no una?** Una canonada és un sol flux: si tots dos extrems llegeixen i escriuen a la mateixa, cadascun pot llegir els seus propis missatges (els bytes els consumeix qui arriba primer a `read`). Amb `toserver` i `toclient` cada sentit té el seu canal.
- **N clients.** Una FIFO d'entrada comuna cap al servidor i una FIFO de sortida per client; el servidor reenvia cada missatge a totes les sortides. Una sola FIFO de sortida no funciona: els clients es «robarien» els missatges.
- **No veure els teus propis missatges.** Cada missatge comença amb un identificador de client (per exemple el seu pid); el servidor no el reenvia al seu autor. Aquest és el repte 1A del laboratori.

Llista de comprovació

Repte amb IA generativa · Xat amb N clients i sense eco dels teus missatges



Objectiu. Resol les preguntes «Per anar més lluny» del TD2: un xat bidireccional amb N clients sobre FIFO on cap client no veu els seus propis missatges.

Prompt inicial. Fes-lo servir a l'eina que prefereixis (Claude, ChatGPT, Gemini, Copilot, Codex...) amb la versió *resolta* del laboratori com a context. Demana primer un pla i revisa cada pas abans d'acceptar-lo.

Prompt

Tinc un servidor i un client de xat en C que fan servir una FIFO (el client escriu, el servidor imprimeix) [enganxa `fifo_server.c` i `fifo_client.c`]. Vull ampliar-lo a N clients que es vegin entre ells: (1) proposa un disseny amb una FIFO d'entrada comuna i una FIFO de sortida per client, i digues-me què falla si tothom llegeix d'una sola FIFO de sortida; (2) cada missatge ha de portar un identificador de client perquè no es mostri a qui l'ha enviat; (3) explica què passa amb les obertures bloquejants de les FIFO i com evitar un interbloqueig en obrir-les. Dona'm el codi pas a pas i espera la meua confirmació entre passos.

Com comprovar el resultat

- Amb 3 terminals, el que escriu cadascun apareix als altres dos i no en ell mateix.
- Tancar un client no bloqueja ni mata els altres.
- Saps explicar per què dos lectors de la mateixa FIFO es «roben» els missatges.

Declara l'ús de la IA: eina, prompts rellevants, què has comprovat tu i què has corregit. El que compta és que puguis explicar i verificar cada decisió.